



Discrete-Event Simulation

Start With a Line You Have Seen

- It is 12:00 and the canteen has one cashier
- Twenty of you arrive over the next ten minutes
- The line grows, shrinks, then grows again
- How long will the next person have to wait?
- Would a second cashier fix it, or waste money?
- Nobody can answer that by looking at the line

Why Guessing Fails Here

- The answer depends on when each person arrives
- It also depends on how long each person takes
- Both of those vary from minute to minute
- An average hides the pile-ups that cause waiting
- A counter looks calm one minute and jammed the next
- We need to follow the line over time, not sample it once

What You Will Learn Today

- The definition of a discrete-event simulation
- The four components every such model contains
- The next-event time-advance algorithm, run by hand
- The same model written in about 15 lines of Python
- Why a single replication is not a result
- How to report an estimate with a confidence interval

Simulation: The Definition

- A model imitates the operation of a real system over time
- Simulation is running that model to observe its behavior
- We experiment on the model, not on the real system
- Change one input, rerun, and compare the outputs
- Our models are built from logic and numbers, not hardware
- **Analogy:** a pilot trains in a flight simulator, not a real plane

Where We Left Off: Monte Carlo

- Last session you sampled random numbers repeatedly
- You used those samples to estimate one quantity
- Each sample was independent of every other sample
- There was no clock and no queue in that model
- The same random number generator drives today's model
- We are adding two things to it: time, and system state

Static Models and Dynamic Models

- A Monte Carlo model is static: it has no time variable
- It estimates how often or how much, never when
- It cannot represent one entity waiting for another
- A discrete-event model is dynamic: state evolves over time
- Queues are about order and timing, not frequency alone
- That difference is the reason this topic exists

Discrete Change and Continuous Change

- In a discrete system, state changes at separate instants
- In a continuous system, state changes smoothly at all times
- Discrete-event simulation models the first kind
- It moves from one state change to the next, skipping the gaps
- Continuous simulation integrates equations over small steps
- **Analogy:** a film shows every frame, a story jumps between scenes

What an Event Is

- An event is an instantaneous occurrence that changes state
- Arrival event: one more customer is now in the system
- Departure event: the server has become free
- Failure event: available capacity has dropped
- Events themselves take no time; the intervals between them do
- Between two events the state is unchanged, so nothing is computed

Discrete or Continuous?

System	State variable	Kind
Canteen queue	number waiting	Discrete
Enrollment window	students in line	Discrete
Water in a tank	height of the water	Continuous
Server room	temperature	Continuous
Toll gate	cars in the lane	Discrete
Moving jeepney	speed	Continuous

The Standard Terms

- **Entity:** an object that moves through the system (a customer)
- **Attribute:** a property held by one entity (the order size)
- **Resource:** a limited server entities compete for (the cashier)
- **Event:** an instant at which state changes (reaching the counter)
- **Activity:** a duration the model draws (paying)
- **Delay:** a duration the model cannot draw (queue waiting)

Activity and Delay

- An activity is an unconditional wait: its length is known at the start
- A delay is a conditional wait: the system decides its length
- Service time is an activity, so the model samples it
- Queue waiting time is a delay, so the model cannot sample it
- Delay is produced by congestion, which is the thing we study
- Delay is always an output of the model, never an input

The Four Components of the Model

- **System state:** variables describing the system at this instant
- **Simulation clock:** the current value of simulated time
- **Future event list:** pending events ordered by event time
- **Statistical counters:** values accumulated for the output report
- **Together these define every discrete-event simulation**
- **Analogy for the event list:** a diary of future appointments

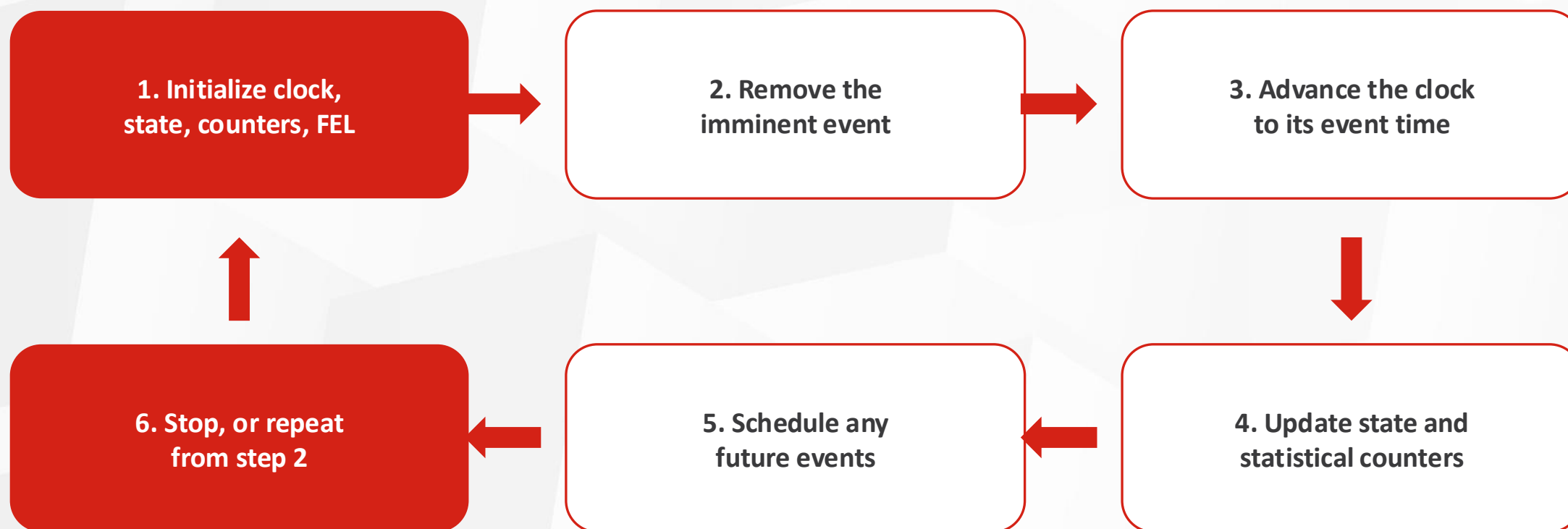
The Future Event List

- The FEL holds events that are scheduled but have not occurred
- Each record carries an event time and an event type
- The list is always kept in order of event time
- The simulation always processes the imminent event, the earliest
- Processing an event may schedule new events onto the list
- Analogy:** a diary you read by jumping to the next appointment

Fixed-Increment and Next-Event

- Fixed-increment time advance steps the clock by a constant
- Next-event time advance jumps to the next event time
- An eight-hour day is 28,800 one-second steps
- With 40 customers, only about 80 of those steps do anything
- On a discrete-event model both give the same answers
- Next-event advance is what SimPy and Arena implement

The Next-Event Time-Advance Algorithm



One Pass Through the Algorithm

- The imminent event is: arrival of customer 2 at time 6
- The clock advances from 5 to 6
- System state changes: the server goes from idle to busy
- A service time is sampled for that customer, say 2 minutes
- A departure event is scheduled onto the FEL at time 8
- Return to step 2, and the FEL determines what happens next

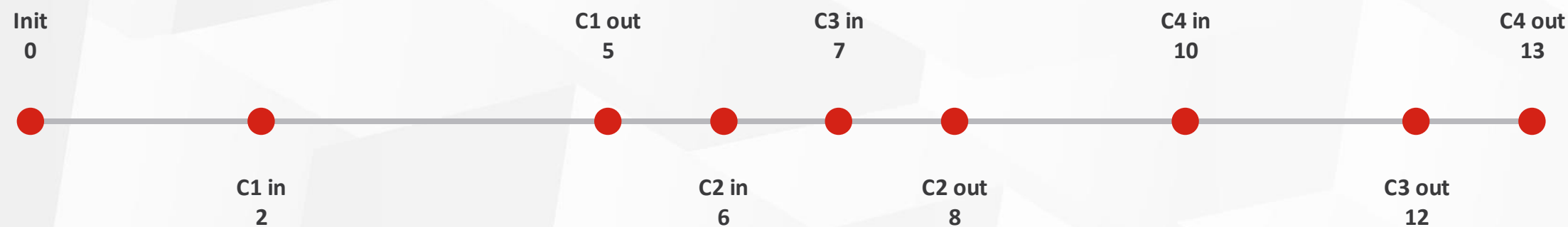
A Model We Can Run by Hand

- Single server, first come first served, unlimited queue
- Interarrival times: 2, 4, 1 and 3 minutes
- Service times: 3, 2, 4 and 1 minutes
- So arrivals occur at times 2, 6, 7 and 10
- The model has exactly two event types
- Arrival and departure. Nothing else changes state

The Trace, Event by Event

Clock	Event	In service	In queue	Scheduled onto FEL
0	Initialize	idle	0	arrival C1 at 2
2	Arrival C1	C1	0	departure at 5
5	Departure C1	idle	0	server now idle
6	Arrival C2	C2	0	departure at 8
7	Arrival C3	C2	1	C3 joins the queue
8	Departure C2	C3	0	departure at 12
10	Arrival C4	C3	1	C4 joins the queue
12	Departure C3	C4	0	departure at 13
13	Departure C4	idle	0	system empty

How the Clock Advances



Nine event times across 13 minutes. Between two events the state does not change.

Reading the Output Off the Trace

- C3 arrived at 7 and began service at 8, so its delay was 1
- C4 arrived at 10 and began service at 12, so its delay was 2
- C1 and C2 entered service on arrival, so their delays were 0
- Total delay 3 minutes over 4 customers gives $Wq = 0.75$
- The server was busy 10 of 13 minutes, so utilization is 0.77
- Both statistics came from the statistical counters, nothing else

Standard Performance Measures

- **W_q** : average delay in the queue, per entity
- **W** : average time in the system, delay plus service
- **L_q** : time-average number of entities in the queue
- **L** : time-average number of entities in the system
- **Utilization**: proportion of time the server is busy
- **Lower case counts the queue, upper case counts the system**

Little's Law

- For a stable system in steady state, $L = \lambda \times W$
- λ is the average arrival rate that enters the system
- The same relation holds for the queue alone: $L_q = \lambda \times W_q$
- It holds for any arrival and service distribution
- Use it to check your output, not to replace the run
- **Analogy:** 2 enter a room a minute, each stays 5, so 10 are inside

Little's Law on Our Trace

- Four entities entered across 13 minutes, so λ is 0.31
- Times in system were 3, 2, 5 and 3 minutes
- So W , the average time in the system, is 3.25 minutes
- Little's Law predicts $L = 0.31 \times 3.25$, which is 1.0
- Integrating the number in system over the run also gives 1.0
- The two routes agree, so the trace was read correctly

Two Simulation World Views

- Event scheduling: write one routine per event type
- You maintain the future event list in your own code
- Process interaction: describe one entity's life cycle
- The library maintains the future event list for you
- Our hand trace used the event-scheduling world view
- SimPy and Arena both use process interaction

Setup: Python and SimPy

```
python -m venv .venv
```

```
.venv\Scripts\activate          # Windows PowerShell
```

```
source .venv/bin/activate       # macOS and Linux
```

```
pip install simpy numpy
```

```
python -c "import simpy; print(simpy.__version__)"
```

```
# 4.1.2
```

Generators and the yield Keyword

- A function containing yield is a generator function
- Calling it returns a generator; the body does not run yet
- yield suspends the function and returns control to the caller
- The generator later resumes on the line after the yield
- A SimPy process is a generator function
- **Analogy:** a bookmark that holds your exact page in the book

SimPy in Nine Lines

```
import simpy

def clock(env, name, tick):
    while True:
        print(name, env.now)
        yield env.timeout(tick)

env = simpy.Environment()
env.process(clock(env, 'tick', 2))
env.run(until=6)

# tick 0
# tick 2
# tick 4
```

Mapping SimPy to the Algorithm

- **Environment:** holds the simulation clock and the FEL
- **A process:** a generator describing one entity's life cycle
- **env.timeout(2):** schedules this process to resume at now plus 2
- **env.process(gen):** places the generator on the FEL
- **env.run(until=6):** executes the algorithm until the clock reaches 6
- **The printed 0, 2, 4 are event times, not clock ticks**

Generating Arrivals

```
import random, simpy

def source(env, mean_gap):
    i = 0
    while True:
        yield env.timeout(random.expovariate(1.0 / mean_gap))
        i += 1
        print(f'customer {i} arrives at {env.now:.2f}')

random.seed(42)
env = simpy.Environment()
env.process(source(env, 2.0))
env.run(until=10)
```

Interarrival Times

- The gap between two consecutive arrivals is the interarrival time
- expovariate samples it from an exponential distribution
- Exponential interarrival times give a Poisson arrival process
- The mean is still the 2 minutes that was requested
- Seed 42 gave 2.04, 2.09, 2.73 and 3.24, then 5.91 and 8.16
- That clustering, not the mean, is what creates a queue

A Single-Server Queue

```
def customer(env, counter, mean_service, waits):
    arrive = env.now
    with counter.request() as turn:
        yield turn                    # queue delay ends here
        waits.append(env.now - arrive) # record the delay
    yield env.timeout(
        random.expovariate(1.0 / mean_service))

def arrivals(env, counter, gap, service, waits):
    while True:
        yield env.timeout(random.expovariate(1.0 / gap))
        env.process(customer(env, counter, service, waits))
```

Seizing and Releasing a Resource

- `request()` asks to seize the resource, which may not be free
- `yield turn` suspends the process until the resource is granted
- The elapsed time across those two lines is the queue delay
- Leaving the with block releases the resource to the next entity
- Without release the resource is never freed and the queue diverges
- **Analogy:** take a number, wait for it to be called, then leave

One Replication

```
def run_once(seed, until=480, gap=2.0, service=1.5):  
    random.seed(seed)  
    env = simpy.Environment()  
    counter = simpy.Resource(env, capacity=1)  
    waits = []  
    env.process(arrivals(env, counter, gap, service, waits))  
    env.run(until=until)  
    return sum(waits) / len(waits)  
  
print(round(run_once(1), 2))  
# 4.24
```

Why One Run Is Not a Result

- One run of a stochastic model is one random sample
- Independent runs with different seeds are called replications
- Each replication is an equally valid realization of the model
- The estimate is the mean across replications, not one run
- That estimate must be reported with a confidence interval
- **Analogy:** one Tuesday at the canteen is not the average Tuesday

Pseudorandom Numbers and the Seed

- The generator is deterministic, not truly random
- It produces a fixed stream of numbers from a starting value
- That starting value is called the seed
- The same seed reproduces the same stream on any machine
- Reporting the seed is what makes your run reproducible
- A different seed produces an independent replication

Ten Replications

```
import statistics

results = [run_once(seed) for seed in range(1, 11)]
mean = statistics.mean(results)
half = 2.262 * statistics.stdev(results) / (10 ** 0.5)

print(f'mean wait {mean:.2f} +/- {half:.2f} minutes')
# mean wait 3.55 +/- 1.35 minutes

# 2.262 is t(0.025, 9), the 95 percent value for 10 runs
```

Reading the Confidence Interval

- 3.55 minutes is the point estimate of the mean delay
- 1.35 is the half-width of the 95 percent confidence interval
- So the interval runs from 2.20 to 4.90 minutes
- A wide interval means more replications are needed
- Half-width shrinks with the square root of the number of runs
- Report the interval; the point estimate alone is not a result

Initialization Bias

- The model starts empty and idle, which is not a typical state
- Early entities meet no queue, so their delays are short
- Those early observations bias the average downward
- This early stretch is called the transient period
- Queueing theory gives $Wq = 4.5$ minutes for these parameters
- We measured 3.55, and initialization bias is the reason

Warm-Up and Steady State

- Steady state is when the output stops drifting with time
- Let the model reach it before collecting any statistics
- Discard the observations from the transient period
- This discarded stretch is the warm-up period
- Choose its length by plotting the running average
- Report the warm-up length beside the run length

Mistakes That Cost Marks

- Reporting a single replication as though it were the answer
- Collecting statistics through the transient period
- Setting a service rate lower than the arrival rate
- Ignoring that utilization near 1 makes delay grow sharply
- Verifying the code but never validating the model
- Omitting the seed, so the work cannot be reproduced

Glossary: Term, Meaning, Analogy

Term	Formal meaning	Analogy used today
Event	Instantaneous change of state	A scene in a story
Future event list	Pending events in time order	A diary of appointments
Entity	Object moving through the system	A canteen customer
Resource	Limited server entities compete for	The cashier
Delay	Conditional wait set by the system	Standing in the line
Replication	One independent run of the model	One Tuesday

Key Takeaways

- Simulation experiments on a model, not on the real system
- An event is an instant of state change; between events, nothing
- The future event list drives the entire run
- SimPy maintains that list; the modelling decisions stay yours
- One replication is a sample; report a confidence interval
- Little's Law is a free check on your own output

Thank you!